

ASSIGNMENT 3.2

Đề bài:

Xét các cấu trúc dữ liệu Singly Linked List, Doubly Linked List, Stack, Queue.

a/ Chỉ ra 02 ứng dụng thực tế mà bạn cho là có giá trị cao nhất của mỗi cấu trúc.

b/ Giải thích rõ tại sao (ứng dụng được nêu có giá trị thực tế cao).

c/ Cho biết cấu trúc tương ứng được dùng như thế nào trong ứng dụng (có thể khai báo và viết một vài đoạn code minh họa)

Giải:

- Singly Linked List:

a, 02 ứng dụng thực tế có giá trị cao nhất:

1. Quản lý danh sách chờ khách tại nhà hàng, quán ăn (Restaurant Waitlist): Lưu trữ danh sách khách đang chờ bàn theo thứ tự.

2. Triển khai bộ nhớ đệm LRU - Least Recently Used Cache: Quản lý các bộ nhớ đệm để ưu tiên giữ lại các mục được dùng gần nhất.

b, Lý do có giá trị thực tế cao:

1. Danh sách chờ khách:

- Giá trị: Trong lúc nhà hàng, quán ăn đông khách, việc quản lý hiệu quả lượng khách giúp đảm bảo họ được phục vụ theo thứ tự một cách công bằng, tăng trải nghiệm và tối ưu hóa vận hành cho doanh nghiệp
- Lý do chọn Singly Linked List là vì khách mới được thêm vào cuối danh sách và người đầu tiên sẽ được phục vụ trước (FIFO – First In First Out). Do đó, thao tác với phần tử ở đầu hoặc cuối có chi phí thấp và danh sách động có thể mở rộng mà không cần cấp phát trước.

2. Bộ nhớ đệm LRU:

- Giá trị: Bộ nhớ đệm LRU được sử dụng rộng rãi trong cơ sở dữ liệu hệ điều hành, hoặc trình duyệt để tối ưu hóa truy cập dữ liệu. Nó đảm bảo dữ liệu được truy cập gần đây nhất được giữ lại nhằm để cải thiện hiệu suất hệ thống.

- Lý do: Singly Linked List có thể được sử dụng để theo dõi thứ tự truy cập (mặc dù thông thường dùng Doubly Linked List, nhưng với cách thiết kế đơn giản, Singly Linked List cũng khả thi). Việc thêm phần tử mới vào đầu danh sách hoặc xóa phần tử cuối (ít được dùng nhất) tương đối dễ dàng.

c, Minh họa:

1. Quản lý danh sách chờ khách:

```
#include <bits/stdc++.h>
using namespace std;

struct Customer {
    string name;
    int groupSize;
    Customer* next;
    Customer(string n, int size) : name(n), groupSize(size), next(nullptr) {}
};

class Waitlist {
private:
    Customer* head;
public:
    Waitlist() : head(nullptr) {}

    // Thêm khách vào cuối danh sách
    void addCustomer(string name, int groupSize) {
        Customer* newCustomer = new Customer(name, groupSize);
        if (!head) {
            head = newCustomer;
            return;
        }
        Customer* temp = head;
        while (temp->next) {
            temp = temp->next;
        }
        temp->next = newCustomer;
    }

    // Gọi người ở đầu tiên
    void callNext() {
        if (!head) {
            cout << "Danh sách trống!\n";
            return;
        }
    }
};
```

```

        Customer* temp = head;
        cout << "Gọi: " << temp->name << " (Nhóm " << temp->groupSize << "
người)\n";
        head = head->next;
        delete temp;
    }

    // Hiển thị danh sách
    void displayWaitlist() {
        Customer* temp = head;
        if (!temp) {
            cout << "Danh sách trống!\n";
            return;
        }
        cout << "Danh sách:\n";
        while (temp) {
            cout << temp->name << " (Nhóm " << temp->groupSize << " người)\n";
            temp = temp->next;
        }
    }
};

int main() {
    Waitlist KFC;
    int num;
    cout << "CHÂN THÀNH CẢM ƠN KFC ĐÃ SỬ DỤNG SẢN PHẨM CỦA TÔI ^^ \n";
    cout << "Nhấn phím để lựa chọn hoạt động:\n";
    cout << "1: Đặt bàn\n";
    cout << "2: Hiển thị danh sách đã đặt bàn hiện tại\n";
    cout << "3: Gọi khách\n";
    cout << "4: Ngưng sử dụng\n";
    while(true){
        cout << "Vui lòng nhập số để sử dụng: ";
        cin >> num;
        if(num == 1){
            int t;
            cout << "Nhập số lượng nhóm đặt: ";
            cin >> t;
            while(t--){
                cin.ignore();
                string name;
                int groupSize;

                cout << "Nhập tên: ";
                getline(cin, name);
            }
        }
    }
}

```

```

        cout << "Nhập số lượng người: ";
        cin >> groupSize;
        KFC.addCustomer(name, groupSize);
    }
}
if(num == 2) KFC.displayWaitlist();
if(num == 3) KFC.callNext();
if(num == 4) return 0;
}
}

```

2. Bộ nhớ đệm LRU:

```

#include <bits/stdc++.h>
using namespace std;

struct CacheEntry
{
    string key;
    int value;
    CacheEntry* next;
    CacheEntry(string k, int v) : key(k), value(v), next(nullptr) {}
};

class LRUCache
{
private:
    CacheEntry* head;
    int capacity;
    int size;

public:
    LRUCache(int cap) : head(nullptr), capacity(cap), size(0) {}

    // Truy cập hoặc thêm mục mới
    void access(string key, int value)
    {
        // Kiểm tra xem key đã tồn tại chưa
        CacheEntry* prev = nullptr;
        CacheEntry* curr = head;
        while (curr && curr->key != key)
        {
            prev = curr;
            curr = curr->next;
        }
    }
}

```

```

    if (curr) {
        // Nếu key tồn tại, di chuyển lên đầu
        if (prev) {
            prev->next = curr->next;
            curr->next = head;
            head = curr;
        }
        // Cập nhật giá trị
        curr->value = value;
        return;
    }

    // Thêm mục mới
    CacheEntry* newEntry = new CacheEntry(key, value);
    newEntry->next = head;
    head = newEntry;
    size++;

    // Nếu vượt quá dung lượng, xóa mục cuối
    if (size > capacity) {
        CacheEntry* temp = head;
        CacheEntry* newLast = nullptr;
        while (temp->next) {
            newLast = temp;
            temp = temp->next;
        }
        if (newLast) {
            newLast->next = nullptr;
            delete temp;
            size--;
        }
    }
}

// Hiển thị
void displayCache() {
    CacheEntry* temp = head;
    cout << "Bộ nhớ đệm (từ mới nhất):\n";
    while (temp) {
        cout << "Key: " << temp->key << ", Value: " << temp->value << "\n";
        temp = temp->next;
    }
}
};

```

```

int main() {
    int num;
    cout << "CẢM ƠN BẠN ĐÃ SỬ DỤNG SẢN PHẨM CỦA TÔI ^^\\n";
    cout << "Nhấn phím để lựa chọn hoạt động:\\n";
    cout << "1: Truy cập hoặc thêm mục mới\\n";
    cout << "2: Hiển thị bộ nhớ đệm\\n";
    cout << "3: Ngưng sử dụng\\n";
    cout << "Vui lòng nhập dung lượng cache: \\n";
    int capacity;
    cin >> capacity;
    LRUCache cache(capacity);
    while(true){
        cout << "Vui lòng nhập số để sử dụng: ";
        cin >> num;
        if(num == 1){
            int t;
            cout << "Nhập số lượt truy cập: ";
            cin >> t;

            while(t--){
                cin.ignore();
                string key;
                int value;
                cout << "Nhập key: ";
                getline(cin, key);
                cout << "Nhập giá trị: ";
                cin >> value;
                cache.access(key, value);
            }
        }
        if(num == 2) cache.displayCache();
        if(num == 3) return 0;
    }
}

```

- Doubly Linked List

a, 02 ứng dụng thực tế có giá trị cao nhất:

1. Quản lý danh sách liên lạc trong ứng dụng nhắn tin (Contact List Navigation): Cho phép duyệt qua danh sách liên lạc theo cả hai hướng.
2. Triển khai trình chỉnh sửa văn bản dòng (Line-based Text Editor): Quản lý các dòng văn bản trong trình soạn thảo như Vim hay Nano.

b, Lý do có giá trị thực tế cao:

1. Danh sách liên lạc trong ứng dụng nhắn tin:

- Giá trị: Trong các ứng dụng như Messenger, Zalo, danh sách liên lạc cần được duyệt nhanh chóng, đặc biệt khi người dùng cuộn lên/xuống để tìm kiếm. Khả năng di chuyển hai chiều giúp cải thiện trải nghiệm người dùng.
- Lý do: Doubly Linked List cho phép truy cập phần tử trước/sau với chi phí $O(1)$, phù hợp để hiển thị danh sách liên lạc dài mà không cần tải lại toàn bộ dữ liệu.

2. Trình chỉnh sửa văn bản dòng:

- Giá trị: Các trình soạn thảo văn bản như Vim, Nano được sử dụng rộng rãi bởi lập trình viên và quản trị hệ thống. Việc thao tác dòng văn bản hoặc di chuyển con trỏ giữa các dòng sẽ được thực hiện nhanh chóng.
- Lý do: Doubly Linked List hỗ trợ thêm/xóa dòng ở bất kỳ vị trí nào và di chuyển con trỏ lên/xuống dễ dàng nhờ con trỏ prev và next, giúp tối ưu hóa hiệu suất so với mảng.

c, Minh họa:

1. Quản lý danh sách liên lạc:

```
#include <iostream>
#include <string>
using namespace std;

struct Contact {
    string name;
    string phone;
    Contact* prev;
    Contact* next;
```

```

    Contact(string n, string p) : name(n), phone(p), prev(nullptr), next(nullptr)
{}
};

class ContactList {
private:
    Contact* head;
    Contact* current; // Con trỏ đến Liên Lạc

public:
    ContactList() : head(nullptr), current(nullptr) {}

    // Thêm Liên Lạc mới
    void addContact(string name, string phone) {
        Contact* newContact = new Contact(name, phone);
        if (!head) {
            head = current = newContact;
            return;
        }
        Contact* temp = head;
        while (temp->next) {
            temp = temp->next;
        }
        temp->next = newContact;
        newContact->prev = temp;
    }

    // Di chuyển lên (liên lạc trước)
    string moveUp() {
        if (!current || !current->prev) {
            return "Đã ở đầu danh sách!";
        }
        current = current->prev;
        return current->name + " (" + current->phone + ")";
    }

    // Di chuyển xuống (liên lạc sau)
    string moveDown() {
        if (!current || !current->next) {
            return "Đã ở cuối danh sách!";
        }
        current = current->next;
        return current->name + " (" + current->phone + ")";
    }
}

```



```

// Hiển thị Liên lạc hiện tại
string getCurrent() {
    return current ? current->name + " (" + current->phone + ")" : "Danh sách trống!";
}
};

int main() {
    ContactList contacts;
    int num;
    cout << "CẢM ƠN BẠN ĐÃ SỬ DỤNG SẢN PHẨM CỦA TÔI ^^\\n";
    cout << "Nhấn phím để lựa chọn hoạt động:\\n";
    cout << "1: Thêm liên lạc\\n";
    cout << "2: Hiển thị liên lạc hiện tại\\n";
    cout << "3: Di chuyển\\n";
    cout << "4: Ngưng sử dụng\\n";
    while(true){
        cout << "Vui lòng nhập số để sử dụng: ";
        cin >> num;
        if(num == 1){
            int t;
            cout << "Nhập số lượng thêm: ";
            cin >> t;
            while(t--){
                cin.ignore();
                string name, phone;
                cout << "Nhập tên: ";
                getline(cin, name);
                cout << "Nhập số liên lạc: ";
                cin >> phone;
                contacts.addContact(name, phone);
            }
        }
        if(num == 2) cout << contacts.getCurrent() << endl;;
        if(num == 3){
            int choice;
            cout << "Nhấn phím 0 để di chuyển xuống\\n";
            cout << "Nhấn phím 1 để di chuyển lên\\n";
            cin >> choice;
            if(choice == 0) cout << contacts.moveDown() << endl;
            else if(choice == 1) cout << contacts.moveUp() << endl;
            else cout << "Vui lòng nhập chỉ 0 hoặc 1!!!\\n";
        }
        if(num == 4) return 0;
    }
}

```

2. Trình chỉnh sửa văn bản dòng:

```
#include <iostream>
#include <string>
using namespace std;

struct Line {
    string text;
    Line* prev;
    Line* next;
    Line(string t) : text(t), prev(nullptr), next(nullptr) {}
};

class TextEditor {
private:
    Line* head;
    Line* current; // Dòng hiện tại
public:
    TextEditor() : head(nullptr), current(nullptr) {}

    // Thêm dòng mới sau dòng hiện tại
    void addLine(string text) {
        Line* newLine = new Line(text);
        if (!head) {
            head = current = newLine;
            return;
        }
        newLine->next = current->next;
        newLine->prev = current;
        if (current->next) {
            current->next->prev = newLine;
        }
        current->next = newLine;
        current = newLine;
    }

    // Xóa dòng hiện tại
    void deleteLine() {
        if (!current) {
            cout << "Không có dòng nào!\n";
            return;
        }
        Line* temp = current;
        if (current->prev) {
            current->prev->next = current->next;
```

```

    } else {
        head = current->next;
    }
    if (current->next) {
        current->next->prev = current->prev;
    }
    current = current->next ? current->next : current->prev;
    delete temp;
}

// Di chuyển lên
void moveUp() {
    if (current && current->prev) {
        current = current->prev;
    }
}

// Di chuyển xuống
void moveDown() {
    if (current && current->next) {
        current = current->next;
    }
}

// Hiển thị văn bản
void displayText() {
    Line* temp = head;
    int lineNum = 1;
    while (temp) {
        cout << lineNum++ << ": " << temp->text << endl;
        temp = temp->next;
    }
    cout << "Dòng hiện tại: " << (current ? current->text : "Trống") << endl;
}
};

int main() {
    TextEditor editor;
    int num;
    cout << "CẢM ƠN BẠN ĐÃ SỬ DỤNG SẢN PHẨM CỦA TÔI ^^\\n";
    cout << "Nhấn phím để lựa chọn hoạt động:\\n";
    cout << "1: Thêm hoặc xóa dòng\\n";
    cout << "2: Hiển thị liên lạc hiện tại\\n";
    cout << "3: Di chuyển\\n";
    cout << "4: Ngưng sử dụng\\n";

```

```

while(true){
    cout << "Vui lòng nhập số để sử dụng: ";
    cin >> num;
    if(num == 1){
        int t;
        cout << "Nhấn phím 0 để xóa dòng\n";
        cout << "Nhấn phím 1 để thêm dòng\n";
        cout << "Vui lòng nhập số để sử dụng: ";
        cin >> t;
        cin.ignore();
        if(t == 1){
            string text;
            cout << "Nhập nội dung: ";
            getline(cin, text);
            editor.addLine(text);
        }
        else if(t == 0) editor.deleteLine();
        else cout << "Vui lòng nhập chỉ 0 hoặc 1!!!\n";
    }
    if(num == 2) editor.displayText();
    if(num == 3){
        int choice;
        cout << "Nhấn phím 0 để di chuyển xuống\n";
        cout << "Nhấn phím 1 để di chuyển lên\n";
        cin >> choice;
        if(choice == 0) editor.moveDown();
        else if(choice == 1) editor.moveUp();
        else cout << "Vui lòng nhập chỉ 0 hoặc 1!!!\n";
    }
    if(num == 4) return 0;
}
}

```

- Stack:

a, 02 ứng dụng thực tế có giá trị cao nhất:

1. Triển khai thuật toán Depth-First Search (DFS): Được sử dụng trong các hệ thống tìm kiếm đường đi (như GPS, trò chơi).
2. Quản lý lời gọi hàm trong hệ điều hành (Call Stack): Theo dõi các hàm được gọi trong quá trình thực thi chương trình.

b, Lý do có giá trị thực tế cao:

1. Thuật toán DFS:

- Giá trị: DFS được sử dụng trong các ứng dụng như tìm đường trong bản đồ (Google Maps), giải câu đố (Sudoku, mê cung), hoặc phát hiện chu trình trong đồ thị. Nó giúp tìm kiếm hiệu quả trong không gian trạng thái lớn.
- Lý do: Stack hỗ trợ cơ chế LIFO, phù hợp với cách DFS khám phá nhánh sâu nhất trước khi quay lại, đảm bảo theo dõi trạng thái một cách tự nhiên và tiết kiệm bộ nhớ.

2. Lỗi gọi hàm (Call Stack):

- Giá trị: Call Stack là thành phần cốt lõi của bất kỳ hệ điều hành hoặc trình biên dịch nào, cho phép thực thi các chương trình phức tạp bằng cách quản lý các lời gọi hàm đệ quy hoặc lồng nhau.
- Lý do: Stack lưu trữ thông tin về các hàm đang thực thi (địa chỉ trả về, tham số, biến cục bộ) theo thứ tự LIFO, đảm bảo hàm gọi sau hoàn thành trước và trả về đúng hàm gọi trước đó.

c, Minh họa:

1. Thuật toán DFS:

```
#include <bits/stdc++.h>
using namespace std;

class Graph {
private:
    int V; // Số đỉnh
    vector<vector<int>> adj; // Danh sách kề
public:
    Graph(int vertices) : V(vertices) {
        adj.resize(V);
    }
    // Thêm cạnh
    void addEdge(int u, int v) {
        adj[u].push_back(v);
        adj[v].push_back(u); // Đồ thị vô hướng
    }
    // DFS sử dụng stack
    void DFS(int start) {
        vector<bool> visited(V, false);
        stack<int> s;
        s.push(start);
        while (!s.empty()) {
```

```

        int vertex = s.top();
        s.pop();
        if (!visited[vertex]) {
            cout << vertex << " ";
            visited[vertex] = true;

            // Thêm các đỉnh kề chưa thăm vào stack
            for (int neighbor : adj[vertex]) {
                if (!visited[neighbor]) {
                    s.push(neighbor);
                }
            }
        }
        cout << endl;
    }
};

int main() {
    int num, vertice;
    cout << "CẢM ƠN BẠN ĐÃ SỬ DỤNG SẢN PHẨM CỦA TÔI ^^\\n";
    cout << "Nhập số đỉnh của đồ thị: ";
    cin >> vertice;
    Graph g(vertice);
    cout << "Nhấn phím để lựa chọn hoạt động:\\n";
    cout << "1: Thêm cạnh\\n";
    cout << "2: Hiển thị\\n";
    cout << "3: Ngưng sử dụng\\n";
    while(true){
        cout << "Vui lòng nhập số để sử dụng: ";
        cin >> num;
        if(num == 1){
            int u, v;
            cout << "Nhập hai đỉnh(cách nhau bởi dấu cách): ";
            cin >> u >> v;
            g.addEdge(u, v);
        }
        if(num == 2){
            int temp;
            cout << "Nhập stack bắt đầu: ";
            cin >> temp;
            g.DFS(temp);
        }
        if(num == 3) return 0;
    }
}

```

2. Lời gọi hàm:

```
#include <bits/stdc++.h>
using namespace std;

struct FunctionCall {
    string funcName;
    string params;
    FunctionCall(string name, string p) : funcName(name), params(p) {}
};

class CallStack {
private:
    stack<FunctionCall> calls;
public:
    // Gọi hàm
    void callFunction(string funcName, string params) {
        calls.push(FunctionCall(funcName, params));
        cout << "Gọi: " << funcName << "(" << params << ")\n";
    }

    // Kết thúc hàm
    void returnFunction() {
        if (calls.empty()) {
            cout << "Stack rỗng!\n";
            return;
        }
        FunctionCall call = calls.top();
        calls.pop();
        cout << "Trả về từ: " << call.funcName << "(" << call.params << ")\n";
    }

    // Hiển thị stack
    void displayStack() {
        stack<FunctionCall> temp = calls;
        cout << "Call Stack (từ đỉnh):\n";
        while (!temp.empty()) {
            FunctionCall call = temp.top();
            cout << call.funcName << "(" << call.params << ")\n";
            temp.pop();
        }
    }
};

int main() {
    CallStack stack;
```

```

int num;
cout << "CẢM ƠN BẠN ĐÃ SỬ DỤNG SẢN PHẨM CỦA TÔI ^^\\n";
cout << "Nhấn phím để lựa chọn hoạt động:\\n";
cout << "1: Gọi hàm\\n";
cout << "2: Kết thúc hàm\\n";
cout << "3: Hiển thị\\n";
cout << "4: Ngưng sử dụng\\n";

while(true){
    cout << "Vui lòng nhập số để sử dụng: ";
    cin >> num;
    if(num == 1){
        string funcName, params;
        cout << "Nhập hàm và biến số: ";
        cin >> funcName >> params;
        stack.callFunction(funcName, params);
    }
    if(num == 2) stack.returnFunction();
    if(num == 3) stack.displayStack();
    if(num == 4) return 0;
}
}

```

- Queue:

a, 02 ứng dụng thực tế có giá trị cao nhất:

1. Quản lý hàng đợi cuộc gọi trong tổng đài (Call Center Queue): Xử lý các cuộc gọi đến theo thứ tự thời gian.
2. Hàng đợi tác vụ trong hệ thống xử lý sự kiện (Event Loop Queue): Được sử dụng trong các ứng dụng như Node.js để xử lý các sự kiện bất đồng bộ.

b, Lý do có giá trị thực tế cao:

1. Quản lý hàng đợi cuộc gọi trong tổng đài:

- Giá trị: Trong các trung tâm chăm sóc khách hàng (như ngân hàng, viễn thông), việc xử lý cuộc gọi theo thứ tự đến đảm bảo công bằng và tăng sự hài lòng của khách hàng.
- Lý do: Queue hoạt động theo cơ chế FIFO, phù hợp với yêu cầu xử lý cuộc gọi tuần tự. Thêm cuộc gọi vào cuối và lấy cuộc gọi ở đầu có chi phí thấp, đảm bảo hiệu quả ngay cả với số lượng lớn.

2. Hàng đợi tác vụ trong hệ thống xử lý sự kiện:

- Giá trị: Trong các hệ thống bất đồng bộ như Node.js, trình duyệt web, hoặc ứng dụng thời gian thực, hàng đợi sự kiện đảm bảo các tác vụ (như phản hồi người dùng, xử lý yêu cầu mạng) được thực thi đúng thứ tự, cải thiện hiệu suất và trải nghiệm.
- Lý do: Queue lưu trữ các sự kiện theo thứ tự đến, cho phép hệ thống xử lý từng sự kiện một cách có tổ chức, đặc biệt trong môi trường đơn luồng.

c, Minh họa:

1. Quản lý hàng đợi cuộc gọi trong tổng đài

```
#include <bits/stdc++.h>
using namespace std;

struct Call {
    string phoneNumber;
    string time;
    Call(string phone, string t) : phoneNumber(phone), time(t) {}
};

class CallCenter {
private:
    queue<Call> calls;
public:
    // Thêm cuộc gọi mới
    void addCall(string phoneNumber, string time) {
        calls.push(Call(phoneNumber, time));
    }

    // Xử lý cuộc gọi đầu tiên
    void handleCall() {
        if (calls.empty()) {
            cout << "Không có cuộc gọi nào!\n";
            return;
        }
        Call call = calls.front();
        cout << "Xử lý cuộc gọi từ " << call.phoneNumber << " lúc " << call.time
        << endl;
        calls.pop();
    }

    // Hiển thị hàng đợi
    void displayQueue() {
        queue<Call> temp = calls;
```

```

        if (temp.empty()) {
            cout << "Hàng đợi trống!\n";
            return;
        }
        cout << "Hàng đợi cuộc gọi:\n";
        while (!temp.empty()) {
            Call call = temp.front();
            cout << call.phoneNumber << " (lúc " << call.time << ")\n";
            temp.pop();
        }
    }
};

int main() {
    CallCenter center;
    int num;
    cout << "CẢM ƠN BẠN ĐÃ SỬ DỤNG SẢN PHẨM CỦA TÔI ^^!\n";
    cout << "Nhấn phím để lựa chọn hoạt động:\n";
    cout << "1: Thêm cuộc gọi mới\n";
    cout << "2: Xử lý cuộc gọi đầu tiên\n";
    cout << "3: Hiển thị\n";
    cout << "4: Ngưng sử dụng\n";

    while(true){
        cout << "Vui lòng nhập số để sử dụng: ";
        cin >> num;
        if(num == 1){
            string phoneNumber, time;
            cout << "Nhập số điện thoại và thời gian: ";
            cin >> phoneNumber >> time;
            center.addCall(phoneNumber, time);
        }
        if(num == 2) center.handleCall();
        if(num == 3) center.displayQueue();
        if(num == 4) return 0;
    }
}

```

2. Hàng đợi tác vụ trong hệ thống xử lý sự kiện:

```
#include <bits/stdc++.h>
using namespace std;

struct Event {
    string type;
    string data;
    Event(string t, string d) : type(t), data(d) {}
};

class EventLoop {
private:
    queue<Event> events;
public:
    // Thêm sự kiện mới
    void addEvent(string type, string data) {
        events.push(Event(type, data));
    }
    // Xử lý sự kiện đầu tiên
    void processEvent() {
        if (events.empty()) {
            cout << "Không có sự kiện nào!\n";
            return;
        }
        Event evt = events.front();
        cout << "Xử lý sự kiện: " << evt.type << " với dữ liệu: " << evt.data <<
endl;
        events.pop();
    }
    // Hiển thị hàng đợi
    void displayQueue() {
        queue<Event> temp = events;
        if (temp.empty()) {
            cout << "Hàng đợi trống!\n";
            return;
        }
        cout << "Hàng đợi sự kiện:\n";
        while (!temp.empty()) {
            Event evt = temp.front();
            cout << evt.type << ": " << evt.data << endl;
            temp.pop();
        }
    }
};
```

```
int main() {
    EventLoop loop;
    int num;
    cout << "CẢM ƠN BẠN ĐÃ SỬ DỤNG SẢN PHẨM CỦA TÔI ^^\\n";
    cout << "Nhấn phím để lựa chọn hoạt động:\\n";
    cout << "1: Thêm sự kiện mới\\n";
    cout << "2: Xử lý sự kiện đầu tiên\\n";
    cout << "3: Hiển thị\\n";
    cout << "4: Ngưng sử dụng\\n";

    while(true){
        cout << "Vui lòng nhập số để sử dụng: ";
        cin >> num;
        if(num == 1){
            string type, data;
            cout << "Nhập số điện thoại và thời gian: ";
            cin >> type >> data;
            loop.addEvent(type, data);
        }
        if(num == 2) loop.processEvent();
        if(num == 3) loop.displayQueue();
        if(num == 4) return 0;
    }
}
```